

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 8
Operating Systems
Topic: Processes Talking Through Pipes (xv6)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 PART 1: RUN, INSIDE XV6 **4**

2.1 Implementation of `user/run.c` 4

2.2 Makefile Integration and Execution Verification 5

2.3 Answers to Required Observation Questions 6

3 PART 2: BIDIRECTIONAL COMMUNICATION (PINGPONG) **7**

3.1 Implementation of `user/pingpong.c` 7

3.2 Execution and Verification 8

3.3 Answers to Required Observation Questions 8

4 PART 3: CONCURRENT PRIME SIEVE (PRIMES) **9**

4.1 Implementation of `user/primes.c` 9

4.2 Execution and Verification 10

4.3 Analysis and Observations 11

5 PART 4: EXTENDED UTILITY: XARGS **12**

5.1 Implementation of `user/xargs.c` 12

5.2 Execution and Verification 13

6 CONCLUSIONS **14**

REFERENCES **15**

1 INTRODUCTION AND GOAL

1.1 Introduction

Inside the xv6 teaching operating system, standard C runtime libraries such as `glibc`, tracing utilities like `strace`, and manual pages do not exist. The user-space execution environment interfaces with the underlying RISC-V hardware solely through a minimalist set of fundamental kernel system calls. Process management and inter-process communication rely directly on primitives exposed in `user/user.h`: `fork`, `exec`, `wait`, `pipe`, `read`, `write`, `close`, `exit`, and `getpid`.

Understanding how userland processes transition from independent task execution to structured, concurrent pipeline architectures provides deep insight into kernel resource accounting, descriptor table isolation, and concurrent process scheduling without user-space runtime abstractions.

1.2 Lab Objective

The primary goals of this laboratory work are:

- implementing `run`, a utility inside xv6 demonstrating process dispatching, program loading, and exit code collection via `fork`, `exec`, and `wait`;
- implementing `pingpong`, a synchronization protocol where parent and child processes bounce a byte bidirectionally using two distinct pipes;
- writing `primes`, Doug McIlroy's concurrent prime sieve pipeline using chained processes connected via pipes;
- managing file descriptor hygiene under strict kernel constraints, preventing file descriptor leaks (`NOFILE`) and avoiding indefinite pipeline hangs caused by unclosed write descriptors;
- observing kernel process management limits (`NPROC`) and recursion constraints enforced by compiler flags (`-Werror`);
- implementing the extended utility `xargs` to read newline-delimited command arguments from standard input and dispatch them via `fork`, `exec`, and `wait`.

2 PART 1: RUN, INSIDE XV6

2.1 Implementation of `user/run.c`

In Linux (Lab 7), process creation and command dispatching were studied using standard POSIX calls (`fork`, `execvp`, `waitpid`). In this section, identical concepts are implemented natively within the unprivileged userland of xv6.

The `run` utility forks a child process. The child process executes the specified command via `exec()`. If `exec()` fails, the child outputs an error and terminates with exit code 1. The parent waits for the child's termination using `wait(&status)` and outputs the returned exit status. If invoked without arguments, `run` defaults to executing `echo hello from the child`.

Listing 2.1: Source code of `user/run.c`

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int pid;
  int status = 0;

  if (argc < 2) {
    // Default test run when invoked without arguments
    char *echo_argv[] = { "echo", "hello", "from", "the", "child", 0 };
    pid = fork();
    if (pid < 0) {
      fprintf(2, "run: fork failed\n");
      exit(1);
    }
    if (pid == 0) {
      printf("child: pid=%d\n", getpid());
      exec("echo", echo_argv);
      fprintf(2, "run: exec echo failed\n");
      exit(1);
    }
    wait(&status);
    printf("parent: child exited with status %d\n", status);
    exit(0);
  }

  // General run invocation: run <command> [args...]
  pid = fork();
  if (pid < 0) {
    fprintf(2, "run: fork failed\n");
    exit(1);
  }

  if (pid == 0) {
    printf("child: pid=%d\n", getpid());
    exec(argv[1], &argv[1]);
    fprintf(2, "run: exec %s failed\n", argv[1]);
    exit(1);
  }

  wait(&status);
  printf("parent: child exited with status %d\n", status);
  exit(0);
```

```
}
```

Figure 2.1: Figure 2.1 – Source code of user/run.c in xv6

2.2 Makefile Integration and Execution Verification

The target `$U/_run\` was registered in the `UPROGS` variable inside the xv6 `Makefile`. The operating system was then compiled and launched under QEMU to evaluate normal execution, argument passing, and error handling:

```
$ make qemu
...
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh

$ run
child: pid=4
hello from the child
parent: child exited with status 0

$ run echo hello from the child
child: pid=5
hello from the child
parent: child exited with status 0

$ run ls
child: pid=6
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 39792
init           2 7 35832
kill           2 8 35584
ln             2 9 35432
ls             2 10 38048
mkdir          2 11 35640
rm             2 12 35632
sh             2 13 56840
stressfs       2 14 36608
usertests      2 15 192800
grind          2 16 50744
wc             2 17 37192
zombie         2 18 35056
run            2 19 36240
pingpong       2 20 36184
primes         2 21 37016
xargs          2 22 36800
parent: child exited with status 0

$ run nosuchprog
child: pid=7
run: exec nosuchprog failed
parent: child exited with status 1
$
```

Figure 2.2: Figure 2.2 – Execution verification of run utility under xv6

2.3 Answers to Required Observation Questions

a) **Three System Calls Used and Linux Comparison:**

- **Calls used in xv6:** `fork()`, `exec()`, and `wait()`.
- **Correspondence to Linux (Lab 7):** In Linux Lab 7, we employed `fork()`, `execvp()` (or `execlp()`), and `waitpid()`.
- **Key differences:** In xv6, `exec(path, argv)` does not parse an environment `$PATH` string, requiring a direct binary name located in the root filesystem. Furthermore, xv6's `wait(&status)` directly copies the integer exit value passed to `exit(status)`, whereas Linux `waitpid` packs termination signals and exit codes into a composite bitmask that must be unwrapped via macros (`WIFEXITED`, `WEXITSTATUS`).

b) **Origin of Output in the `echo` Case:** The `echo` program prints `"hello from the child"`, running within the address space of the **child process**. When the child invokes `exec("echo", ...)`, the kernel replaces the child process's text and data memory pages with the binary of `echo`. The process retains its PID (e.g., PID 4) and file descriptors, running `echo`'s `main()`, which writes its arguments to descriptor 1 (`stdout`) before terminating.

c) **Exit Status Comparison (`nosuchprog`):**

- When `exec()` fails in xv6 because the requested program does not exist, the kernel returns `-1` to the child process. The child code detects this and explicitly invokes `exit(1)`. Consequently, the kernel records exit code `1`, which the parent retrieves via `wait(&status)`.
- Under Linux, POSIX standards and shell conventions reserve exit status `127` specifically to denote "command not found" (and `126` for "command found but not executable"). In Lab 7, `myrun.c` followed this standard by calling `exit(127)` upon `execvp` failure. xv6 maintains a simple integer convention where `1` indicates a generic error.

3 PART 2: BIDIRECTIONAL COMMUNICATION (PINGPONG)

3.1 Implementation of `user/pingpong.c`

In xv6 and standard UNIX semantics, a pipe is strictly unidirectional. Bidirectional communication requires two separate pipes: one for parent-to-child transmission (`p2c`) and another for child-to-parent transmission (`c2p`). The child reads a single byte from `p2c`, prints its PID along with the confirmation message, and forwards the byte back via `c2p` before exiting.

Listing 3.1: Source code of `user/pingpong.c`

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    int p2c[2]; // parent -> child: [0] read, [1] write
    int c2p[2]; // child -> parent: [0] read, [1] write
    char byte = 'x';

    if (pipe(p2c) < 0 || pipe(c2p) < 0) {
        printf("pipe failed\n");
        exit(1);
    }

    int pid = fork();
    if (pid < 0) {
        printf("fork failed\n");
        exit(1);
    }

    if (pid == 0) {
        // Child process: close unused ends
        close(p2c[1]);
        close(c2p[0]);

        read(p2c[0], &byte, 1);
        printf("%d: received ping\n", getpid());
        close(p2c[0]);

        write(c2p[1], &byte, 1);
        close(c2p[1]);

        exit(0);
    } else {
        // Parent process: close unused ends
        close(p2c[0]);
        close(c2p[1]);

        write(p2c[1], &byte, 1);
        close(p2c[1]);

        read(c2p[0], &byte, 1);
        printf("%d: received pong\n", getpid());
        close(c2p[0]);

        wait(0);
        exit(0);
    }
```

```
50 }
```

Figure 3.1: Figure 3.1 – Source code of user/pingpong.c in xv6

3.2 Execution and Verification

After registering `$U/_pingpong\` inside the `UPROGS` section of the `Makefile`, the kernel was built and launched inside QEMU:

```
1 $ pingpong
2 4: received ping
3 3: received pong
4 $
```

Figure 3.2: Figure 3.2 – Execution trace of pingpong inside xv6

3.3 Answers to Required Observation Questions

a) **Requirement for Two Independent Pipes:** A UNIX pipe provides a single unidirectional byte stream. If both processes shared a single pipe descriptor pair, a severe race condition would emerge: the parent writing the byte into the pipe could immediately execute a subsequent `read()` and consume its own transmitted byte before the child process is scheduled by the kernel. Allocating two distinct channels guarantees isolation and prevents self-reads.

b) **Implicit Synchronization via Blocking Reads:** The source code contains no explicit polling loops or busy-waiting. Synchronization is achieved naturally through the blocking semantics of the `read()` system call. When the parent invokes `read(c2p[0], ...)`, the calling process is transitioned into a sleeping state by the xv6 scheduler because `c2p` contains no data. The parent remains suspended until the child executes `write(c2p[1], ...)`, which wakes the sleeping parent process.

4 PART 3: CONCURRENT PRIME SIEVE (PRIMES)

4.1 Implementation of `user/primes.c`

Doug McIlroy's pipeline model calculates prime numbers by generating an autonomous pipeline of processes. The root process generates integers from 2 through 35 and pushes them into an initial pipe. Each downstream stage reads its first available integer, which is mathematically guaranteed to be prime. The stage prints the value, creates a new pipe, forks a successor stage, and filters remaining integers: values non-divisible by its assigned prime are forwarded, while multiples are dropped.

Listing 4.1: Source code of `user/primes.c`

```c
#include "kernel/types.h"
#include "user/user.h"

void stage(int left_fd) __attribute__((noreturn));

void
stage(int left_fd)
{
    int p;
    int n;

    int bytes = read(left_fd, &p, sizeof(p));
    if (bytes <= 0) {
        close(left_fd);
        exit(0);
    }

    printf("prime %d\n", p);

    int right_pipe[2];
    if (pipe(right_pipe) < 0) {
        close(left_fd);
        exit(1);
    }

    int pid = fork();
    if (pid < 0) {
        close(left_fd);
        close(right_pipe[0]);
        close(right_pipe[1]);
        exit(1);
    }

    if (pid == 0) {
        // Child stage: consume from right_pipe[0]
        close(left_fd);
        close(right_pipe[1]);
        stage(right_pipe[0]);
    } else {
        // Current stage: forward non-divisible numbers to right_pipe[1]
        close(right_pipe[0]);

        while (read(left_fd, &n, sizeof(n)) > 0) {
            if (n % p != 0) {
                write(right_pipe[1], &n, sizeof(n));
            }
        }

```

```
        close(left_fd);
        close(right_pipe[1]); // Close write end to transmit EOF
        wait(0);
        exit(0);
    }
}

int
main(int argc, char *argv[])
{
    int p[2];
    if (pipe(p) < 0) {
        exit(1);
    }

    int pid = fork();
    if (pid < 0) {
        exit(1);
    }

    if (pid == 0) {
        close(p[1]);
        stage(p[0]);
    } else {
        close(p[0]);
        for (int i = 2; i <= 35; i++) {
            write(p[1], &i, sizeof(i));
        }
        close(p[1]);
        wait(0);
        exit(0);
    }
}
```

Figure 4.1: Figure 4.1 – Source code of user/primes.c in xv6

4.2 Execution and Verification

Adding `$U/_primes\` to the `Makefile` and executing under QEMU yielded the complete sequence of primes from 2 to 31:

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
$
```

Figure 4.2: Figure 4.2 – Execution output of primes sieve in xv6

4.3 Analysis and Observations

a) **Process Count at Execution Milestone:** When `"prime 31"` is output to the terminal, exactly 12 processes exist concurrently:

$$N_{\text{active}} = N_{\text{main}} + N_{\text{primes}} = 1 + 11 = 12 \tag{4.1}$$

where N_{main} is the initial generator process, and N_{primes} corresponds to the 11 discovered primes in the range $[2, 35]$: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, and 31.

b) **Upper Limit Scaling (Limit 280 Experiment):** In the interval $[2, 280]$, there are 59 prime numbers. In xv6, `kernel/param.h` fixes `NPROC = 64` and `NOFILE = 16`. With `init`, `sh`, and the initial generator process active, spawning 59 sieve stages requires 62 process slots, approaching the system ceiling. If all unused file descriptors are meticulously closed, the pipeline successfully finishes on stock xv6. However, if file descriptors are leaked, processes hit `NOFILE` (16 file descriptors) long before process limits are reached, resulting in failed `pipe()` invocations and early termination.

c) **Parallelism Characteristics:** This architecture constitutes a parallel pipeline sieve because each stage functions as an independent concurrent filtering worker connected asynchronously via memory-buffered pipes.

5 PART 4: EXTENDED UTILITY: XARGS

5.1 Implementation of `user/xargs.c`

The `xargs` utility reads input from standard input line by line and executes the target utility for each row, appending the parsed line as command-line arguments. In xv6, no environment `PATH` exists, necessitating direct binary invocation via `exec`.

Listing 5.1: Source code of `user/xargs.c`

```c
#include "kernel/types.h"
#include "kernel/param.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    char *args[MAXARG];
    char buf[512];
    char ch;
    int i;

    if (argc < 2) {
        printf("Usage: xargs <command> [args...]\n");
        exit(1);
    }

    // Populate initial command arguments
    for (i = 1; i < argc; i++) {
        args[i - 1] = argv[i];
    }

    int base_argc = argc - 1;
    int pos = 0;

    // Read characters from standard input (file descriptor 0)
    while (read(0, &ch, 1) > 0) {
        if (ch == '\n') {
            buf[pos] = '\0';
            args[base_argc] = buf;
            args[base_argc + 1] = 0;

            int pid = fork();
            if (pid < 0) {
                printf("xargs: fork failed\n");
                exit(1);
            }

            if (pid == 0) {
                exec(args[0], args);
                printf("xargs: exec %s failed\n", args[0]);
                exit(1);
            }

            wait(0);
            pos = 0;
        } else {
            if (pos < (int)sizeof(buf) - 1) {
                buf[pos++] = ch;
            }
        }
```

```
52      }
53
54      // Flush any trailing line missing a newline character
55      if (pos > 0) {
56          buf[pos] = '\0';
57          args[base_argc] = buf;
58          args[base_argc + 1] = 0;
59
60          int pid = fork();
61          if (pid == 0) {
62              exec(args[0], args);
63              exit(1);
64          }
65          wait(0);
66      }
67
68      exit(0);
69  }
```

Figure 5.1: Figure 5.1 – Source code of user/xargs.c in xv6

5.2 Execution and Verification

The command execution flow was verified against the requirement specified in the lab manual:

```
1  $ echo hello too | xargs echo bye
2  bye hello too
3  $
```

Figure 5.2: Figure 5.2 – Verification of xargs pipeline execution

6 CONCLUSIONS

This laboratory work explored inter-process communication and concurrent scheduling inside the xv6 operating system using bare kernel system calls. Implementing low-level pipe primitives directly without runtime library abstractions provided practical insight into operating system resource isolation and scheduling mechanics.

Key conclusions derived from the experiments include:

- **Process Dispatching and Status Propagation:** The `run` utility clarified how parent processes control execution via `fork()`, transfer address spaces through `exec()`, and accurately capture child termination codes with `wait(&status)`.

- **Descriptor Table Isolation:** Pipes are unidirectional byte buffers. Full-duplex messaging between parent and child processes requires two separate pipes to prevent self-consumption and race conditions.

- **Blocking Synchronization and EOF Mechanics:** A reading process blocks automatically when an incoming pipe is empty. Crucially, an end-of-file condition (`EOF`) is only raised when every write end referencing the pipe across the entire system is closed; omitting closures leads directly to deadlocks.

- **Kernel Table Limits:** Concurrent pipeline architectures are heavily bounded by process table capacity (`NPROC`) and per-process descriptor limits (`NOFILE`). Rigorous file descriptor cleanup is essential for sustained system stability.

REFERENCES

[1] Russ Cox, Frans Kaashoek, and Robert Morris, *xv6: a simple, Unix-like teaching operating system*, MIT CSAIL, 2024.

[2] M. Doug McIlroy, *A Research UNIX Reader: System Architecture, 1972-1989*, AT&T Bell Laboratories Computing Science Technical Report 139, 1987.

[3] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books, 2018.

[4] Department of Software Engineering and Automation, *Operating Systems Lab 8: Processes Talking Through Pipes (xv6)*, FCIM, Technical University of Moldova, 2026.